QA → .NET Upskilling: 1-Day Full-Stack Assignment

Oct 23, 2026 ·

Overview

Each engineer builds **BugBoard**, a small defect tracker: a React SPA that calls an ASP.NET Core Web API secured with JWT bearer tokens. It is a solo task with a one-day time box (about 7 focused hours). The domain is one QA engineers already know, so the effort goes into the stack rather than the business rules.

By the end of the day each engineer should be able to:

- Build a REST API in C# with DTOs, validation and correct HTTP status codes
- Secure endpoints with JWT authentication and role-based authorization
- Build a React + TypeScript SPA that logs in, calls a protected API and handles 401/403
- Apply their QA strength by testing their own code

The group has mixed experience, so scope is tiered: **Core** (everyone must finish), **Expected** (most should finish) and **Stretch** (stronger engineers). Core is graded first; Stretch work never makes up for missing Core items.

Before the day

Everything below must be done and checked the day before. Installing tools on the day typically costs one to two hours of a seven-hour budget.

- ☐ .NET 10 SDK (LTS) installed: `dotnet --version` shows 10.x
- ☐ HTTPS dev certificate trusted: `dotnet dev-certs https --trust`
- ☐ Node.js LTS and npm installed: `node -v`, `npm -v`
- ☐ IDE ready: Visual Studio 2026, JetBrains Rider, or VS Code with the C# Dev Kit
- ☐ Git set up and an empty repo created on the company's Git host
- ☐ HTTP client ready: `.http` files in the IDE, Postman, or Bruno
- ☐ Smoke test: `dotnet new webapi -o Smoke` and `npm create vite@latest smoke -- --template react-ts` both run and start

Pre-reading (about 2 hours, recommended for anyone without app-development experience):

- Tutorial: Create a minimal API with ASP.NET Core
- Configure JWT bearer authentication in ASP.NET Core
- Thinking in React and Managing State
- Introduction to JSON Web Tokens

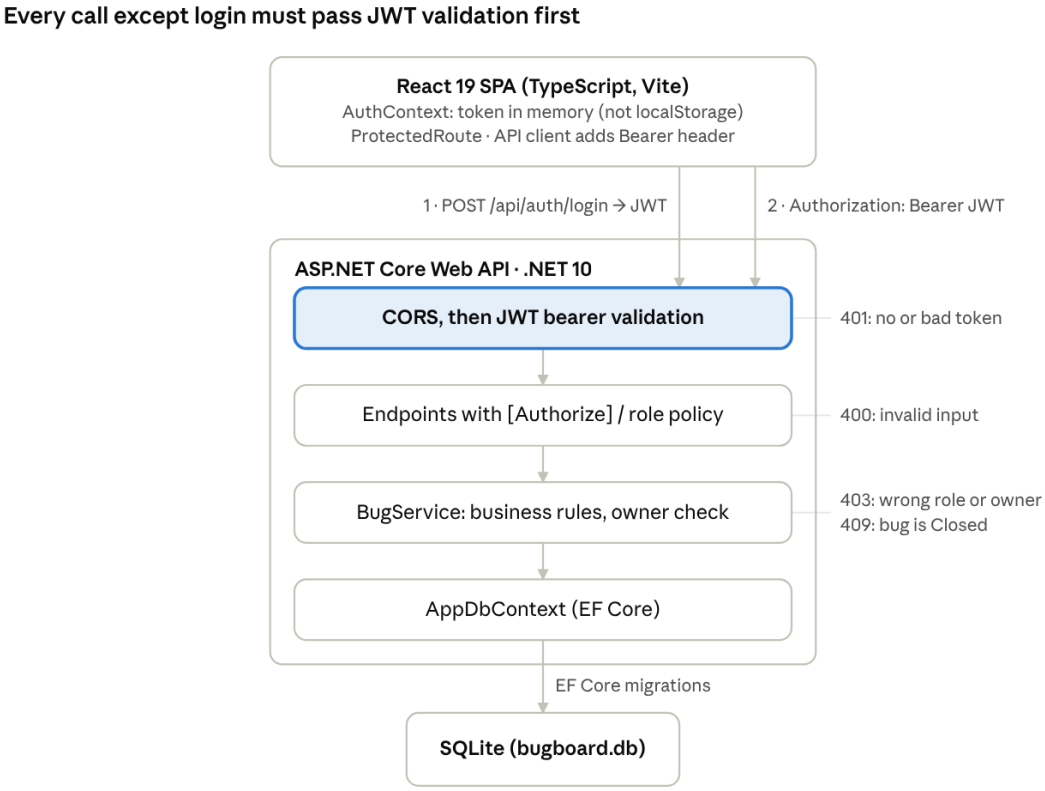
Tech stack and constraints

The stack is fixed so reviews compare like with like. The API issues its own tokens; an external identity provider such as Okta is out of scope for one day.

Layer	Use	Notes
Backend	ASP.NET Core Web API, .NET 10 (LTS), C#	Controllers or minimal APIs: pick one, don't mix
Auth	`Microsoft.AspNetCore.Authentication.JwtBearer`, HS256 tokens issued by the API	Passwords hashed with `PasswordHasher<T>`
Data	Core: in-memory list in a singleton service. Expected: EF Core + SQLite with a migration	SQLite needs no server install
API docs	Built-in OpenAPI (`Microsoft.AspNetCore.OpenApi`) + Scalar or Swagger UI	.NET 9+ templates no longer include Swashbuckle
Frontend	React 19 + TypeScript, Vite, React Router	Not Create React App (deprecated)
HTTP client	`fetch` or axios, in one API module	No API calls scattered across components
Styling	Any; plain CSS is fine	Looks are not graded
Tests	xUnit + `WebApplicationFactory`; Vitest + React Testing Library; Playwright (Stretch)	

Hard constraints:

- **Token in memory only.** Keep the JWT in React state or context, never `localStorage` or `sessionStorage`, because any XSS can read those. Re-login after a page refresh is acceptable today.
- **No secrets in the repo.** The JWT signing key comes from `dotnet user-secrets` or an environment variable.
- **CORS locked down.** Allow only the Vite dev origin, not `AllowAnyOrigin`.
- **The server enforces rules.** Hiding a button in React is UX, not security; every rule is checked in the API.



BugBoard architecture · request path and error codes

Each request passes through the layers top to bottom; the status code on the right is the one that layer is responsible for returning.

Backend requirements

The API exposes login plus CRUD on bugs, with two roles: **Tester** and **Lead**.

Data model

Entity	Fields	Rules
User	Id, Email, DisplayName, PasswordHash, Role	Role is `Tester` or `Lead`. Seed 2 Testers and 1 Lead with known dev passwords.
Bug	Id, Title, Description, Severity, Status, ReportedById, AssignedToId?, CreatedAt, UpdatedAt	Title required, 5–120 chars. Description up to 2,000 chars. Severity: Low, Medium, High, Critical. Status: Open, InProgress, Resolved, Closed. Timestamps in UTC.

API contract

Method	Route	Who	Success	Errors to handle
POST	`/api/auth/login`	Anyone	200 `{ accessToken, expiresAt, user }`	400, 401
GET	`/api/auth/me`	Signed in	200 current user	401
GET	`/api/users`	Signed in	200 list for the assignee dropdown	401
GET	`/api/bugs?status=&severity=&search=&page=&pageSize=`	Signed in	200 paged list with total count	400, 401
GET	`/api/bugs/{id}`	Signed in	200	401, 404
POST	`/api/bugs`	Signed in	201 + `Location` header	400, 401
PUT	`/api/bugs/{id}`	Reporter or Lead	200	400, 401, 403, 404, 409
DELETE	`/api/bugs/{id}`	Lead only	204	401, 403, 404

Business rules

- Only the reporter or a Lead may edit a bug (403 otherwise).
- Only a Lead may set status to Closed or delete a bug.
- A Closed bug cannot be edited: return 409 Conflict.
- Return DTOs only. Never return an entity, so `PasswordHash` can never leak.
- Validation errors return 400 as `ValidationProblemDetails` with a per-field `errors` map.
- Unhandled exceptions return 500 `ProblemDetails` with no stack trace.

Authentication and authorization

- Login checks the hash with `PasswordHasher<User>`. Wrong email and wrong password return the same 401 message, so the API does not reveal which emails exist.
- Tokens: HS256, signing key at least 32 bytes, 60-minute lifetime. Claims: `sub`, `email`, `name`, `role`.
- Validate issuer, audience, lifetime and signing key; all four set in `TokenValidationParameters`.
- Every bug and user endpoint requires authentication by default (a fallback policy or a route group with `RequireAuthorization()`).
- Lead-only actions use a named policy (for example `LeadOnly`). The reporter-or-Lead rule is checked in the service against the current user's `sub` claim.

Frontend requirements

The SPA has four screens behind a login, and it must behave correctly when the API says no.

Screens

Screen	Must do
Login	Email + password; submit disabled while the request runs; one generic error on 401
Bug list	Table of title, severity, status, assignee, updated date; filter by status and severity; search by title; paging (Expected)
Bug detail / edit	Form with client-side validation matching the API rules; server 400 errors shown next to the right fields; 409 shown as a clear message
New bug	Same form; on 201, go to the new bug's page
Lead actions	Close and Delete buttons shown only to Leads, with a confirm step before delete

Auth handling

- An `AuthContext` holds the token and user in memory and exposes `login` and `logout`.
- One API client module adds `Authorization: Bearer <token>` to every request.
- A `ProtectedRoute` sends signed-out users to `/login`.
- A 401 from any call clears the session and returns to login. A 403 shows a "you don't have permission" message and keeps the user where they are.
- Logout clears all auth state.

General

- The API base URL comes from `.env` (`VITE_API_URL`), not hard-coded.
- Every data fetch shows loading, error and empty states.
- Types for API responses live in one place (`types.ts` or similar).

Tiered scope

Finish every Core item before starting Expected, and every Expected item before Stretch. Tick items off as you go; the reviewer uses this list at the demo.

Core (everyone, about 5 hours)

- ☐ Login endpoint issues a valid JWT for seeded users
- ☐ Bug list, get, create and update endpoints, all requiring a token
- ☐ Delete restricted to Lead (403 for Tester)
- ☐ Input validation returning 400 with field errors
- ☐ OpenAPI UI working, with a way to send the bearer token
- ☐ React login, bug list, new bug and edit screens
- ☐ Token kept in memory; 401 sends the user back to login
- ☐ README with run steps and seed credentials

Expected (most engineers, about 1.5 hours)

- ☐ EF Core + SQLite with an initial migration and seed data
- ☐ Filters, search and paging on the list endpoint and screen
- ☐ Server validation errors mapped onto form fields
- ☐ One Playwright end-to-end test: login, create bug, see it in the list